

GitHub webhook értesítések

- Project: Sportmate (ChatGPT)
- Storage: ChatGPT read_thread export
- Session ID: 6abe5301-8c7c-83ed-bc77-9bfe3281be71
- Started: 2026-10-01T12:33:06.738Z
- Updated: 2026-10-01T12:33:06.738Z

This PDF is a classified, derived reading view. Raw JSONL remains the canonical lossless representation when included.

Export note

Forrás: a Sportmate ChatGPT-projekt teljes, egyfordulós beszélgetése. A hivatkozások sorszámai megmaradtak; az eredeti cél-URL-eket a csevegésolvasó nem adta vissza.

User – 2026-10-01T12:34:23.329Z

A Github api ad arra lehetőséget, hogy küldjön nekem egy webhookot, vagy valamilyen értesítést abban az esetben ha egy repository tartalma frissül, pl új commit, issues, pull requests, stars stb.

A célom az lenne, hogy az applikációmban ne egy cronjob syncelgesse a repository adatait, hanem akkor kérjük le ténylegesen, amikor érdemi változás történt.

Final report – 2026-10-01T12:34:53.098Z

Igen. A GitHub Webhooks pontosan erre való. A GitHub HTTP `POST` kérést küld a te endpointodra, amikor a repositoryban kiválasztott esemény történik. [eredeti hivatkozás 1]

A Sportmate esetén ezt az architektúrát használnám:



Milyen változásokról kapsz webhookot?

A fontosabbak:

- | GitHub esemény | Webhook event |
- | ---|---|
- | új commit / push | `push` |
- | issue létrehozás/módosítás/lezárás | `issues` |
- | pull request | `pull\_request` |
- | star hozzáadás/eltávolítás | `star` |
- | branch/tag létrehozás | `create` |
- | branch/tag törlés | `delete` |
- | release | `release` |
- | fork | `fork` |
- | repository metadata változás | `repository` |

A `push` többek között commit push esetén érkezik. A `star` külön `created` és `deleted` actiont küld. [eredeti hivatkozás 2]

Például:

```
POST /api/webhooks/github

X-GitHub-Event: push
X-GitHub-Delivery: 0b989ba4-...
X-Hub-Signature-256: sha256=...
Content-Type: application/json
```

A payloadban többek között ott van a repository és az eseményhez tartozó adat.

```
{
  "ref": "refs/heads/main",
  "before": "abc123",
  "after": "def456",
  "repository": {
    "id": 123456,
    "full_name": "ris/sportmate"
  }
}
```

Nem kell minden webhooknál teljes repository sync

Érdemes eseménytípus alapján célzottan frissíteni:

```
match ($event) {
  'push' => SyncRepositoryCommits::dispatch($repository),
  'issues' => SyncRepositoryIssues::dispatch($repository),
  'pull_request' => SyncRepositoryPullRequests::dispatch($repository),
  'star' => SyncRepositoryStats::dispatch($repository),
  'release' => SyncRepositoryReleases::dispatch($repository),
  default => null,
};
```

Vagy még egyszerűbben:

```
SyncGitHubRepository::dispatch(
  repositoryId: $repository->id,
  reason: $event,
);
```

Így:

```
régen:

cron
↓
100 repository lekérése
↓
99-ben semmi változás

webhookkal:

GitHub változás
↓
1 webhook
↓
1 repository sync
```

Ez jelentősen csökkenti a GitHub API-hívások számát.

Repository webhook vagy GitHub App?

Ha ez csak a ****saját repositoryidhoz**** kell:

```
Repository
Settings
→ Webhooks
→ Add webhook
```

Payload URL:

```
https://sportmate.example/api/webhooks/github
```

Majd kiválasztod:

```
push
issues
pull_request
star
release
...
```

Ha viszont a Sportmate-ben ****felhasználók kapcsolják majd össze saját GitHub repositoryikat****, akkor inkább ****GitHub App****.

Ennek nagy előnye, hogy a GitHub Appnak egyetlen webhook endpointja lehet, és automatikusan megkapja az eseményeket minden olyan repositoryból, amelyhez az adott installation hozzáfér. Repositorynként nem kell külön webhookot létrehozni. [eredeti hivatkozás 3]

Tehát nálad valószínűleg:

```
User
|
▼
Install Sportmate GitHub App
|
├─ repository A
├─ repository B
└─ repository C
    |
    ▼
    GitHub App webhook
    |
    ▼
    https://sportmate.app/api/webhooks/github
```

Ez skálázhatóbb, mint repositorynként külön hook kezelése.

Laravelben fontos: webhook signature

Ne fogadj el vakon bármilyen POST-ot.

GitHub a configured secret alapján küldi:

```
X-Hub-Signature-256: sha256=...
```

A szerveren:

```

$payload = $request->getContent();

$expected = 'sha256=' . hash_hmac(
    'sha256',
    $payload,
    config('services.github.webhook_secret')
);

if (! hash_equals(
    $expected,
    $request->header('X-Hub-Signature-256', '')
)) {
    abort(401);
}

```

GitHub kifejezetten a `X-Hub-Signature-256` HMAC-SHA256 ellenőrzést ajánlja. [eredeti hivatkozás 4]

Az `.env`:

```
GITHUB_WEBHOOK_SECRET=valamilyen-hosszu-random-secret
```

Queue-ba tenném

A webhook controller ne kezdjen több GitHub API requestbe.

```

public function __invoke(Request $request)
{
    // signature validation

    HandleGitHubWebhook::dispatch(
        event: $request->header('X-GitHub-Event'),
        deliveryId: $request->header('X-GitHub-Delivery'),
        payload: $request->json()->all(),
    );

    return response()->noContent();
}

```

GitHub azt ajánlja, hogy a webhook endpoint ****10 másodpercen belül `2XX` választ adjon****, és a hosszabb feldolgozás menjen aszinkron queue-ba. [eredeti hivatkozás 5]

Nálad ez jól illeszkedik a már futó:

```
php artisan queue:listen
```

folymathoz.

`X-GitHub-Delivery`-t mentsd el

Minden delivery kap egy egyedi ID-t:

```
X-GitHub-Delivery: 0b989ba4-242f-11e5-81e1-c7b6966d2516
```

Érdemes adatbázisban unique-ként kezelni:

```
github_webhook_deliveries

id
delivery_id UNIQUE
event
repository_id
processed_at
payload
```

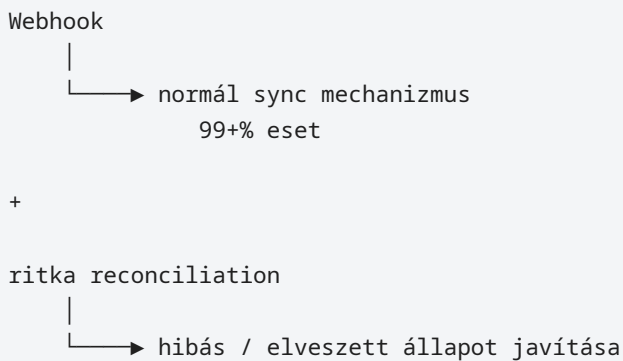
Így egy redelivery nem dolgozódik fel kétszer. GitHub is ezt a headert javasolja deduplikáció/replay-védelem céljára. [eredeti hivatkozás 6]

Egy fontos hátrány

Nem javasolnék **100%-ban webhook-only konzisztenciamodellt**.

GitHub jelenlegi dokumentációja szerint a sikertelen webhook deliveryket **nem küldi automatikusan újra**. A deliveryk manuálisan vagy API-val redeliverelhetők; jelenleg az előző **3 nap** deliveryi redeliverelhetők. [eredeti hivatkozás 7]

Ezért optimális:



Nem kell 5 percenként repositorykat pollolni. Lehet például napi egy reconciliation, vagy csak sikertelen webhook deliveryk ellenőrzése.

Sportmate-hez ezt választanám:

```
GitHub App
  ↓
webhook endpoint
  ↓
signature validation
  ↓
X-GitHub-Delivery deduplication
  ↓
Laravel Queue
  ↓
event-specifikus sync
  ↓
DB
```

Így a cron alapú repository polling gyakorlatilag megszüntethető; a GitHub API-t csak tényleges változáskor használod.

[GitHub webhook events dokumentáció \(https://docs.github.com/en/webhooks/webhook-events-and-payloads?utm_source=chatgpt.com\)](https://docs.github.com/en/webhooks/webhook-events-and-payloads?utm_source=chatgpt.com)

[GitHub webhook best practices \(https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks?utm_source=chatgpt.com\)](https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks?utm_source=chatgpt.com)

[GitHub App webhooks \(https://docs.github.com/en/apps/creating-github-apps/registering-a-github-app/using-webhooks-with-github-apps?utm_source=chatgpt.com\)](https://docs.github.com/en/apps/creating-github-apps/registering-a-github-app/using-webhooks-with-github-apps?utm_source=chatgpt.com)